

Ayuda con Clases y Clima

<https://gemini.google.com/app/5f52a1a54b945688>

User prompt: buenos días, tengo el siguiente proyecto a realizar, me podrias ayudar a realizarlo, me confunde un poco que muchas de las definiciones de clase no especifica que tienen que realizar algunas de sus funciones, despues en una clase uno de los atributos es una clase Clima pero no esta la definicion de la clase clima, podrias ayudarme?

Response: Es completamente normal que sientas dudas al ver un enunciado de programación con cierta flexibilidad. En los proyectos académicos de programación orientada a objetos (POO), los profesores suelen dejar abiertas algunas implementaciones internas para que el alumno tome decisiones de diseño.

A continuación te explico cómo resolver el problema de la clase Clima y te doy las especificaciones exactas para las funciones que te generan confusión.

1. La clase Clima (¿Cómo implementarla si no está definida?)

El enunciado menciona un atributo Clima `climaActual` y varios climas posibles: **Soleado, Lluvioso, Sequía e Invierno**, cada uno con modificadores de rendimiento para las entidades.

La forma más limpia, idiomática y profesional en Java para resolver esto es usar un enum (**Enumeración**) en lugar de una clase tradicional. Un enum te permite definir tipos fijos y agregarles atributos o métodos.

Ejemplo de implementación del enum Clima:

```
public enum Clima {
    SOLEADO(1.5, 5, 0, 1.0),
    LLUVIOSO(2.0, 3, -5, 1.0),
    SEQUIA(0.5, -5, 0, 1.0),
    INVIERNO(0.0, -8, 0, 1.2); // +20% éxito en caza = multiplicador 1.2

    private final double modReproduccionPlanta;
    private final double modEnergiaConejo;
    private final double modEnergiaLobo;
    private final double modExitoCazaLobo;

    Clima(double modReproduccionPlanta, double modEnergiaConejo, double modEnergiaLobo, double modExitoCazaLobo) {
        this.modReproduccionPlanta = modReproduccionPlanta;
        this.modEnergiaConejo = modEnergiaConejo;
        this.modEnergiaLobo = modEnergiaLobo;
        this.modExitoCazaLobo = modExitoCazaLobo;
    }

    // Getters para usar los modificadores durante cada turno
    public double getModReproduccionPlanta() { return modReproduccionPlanta; }
    public double getModEnergiaConejo() { return modEnergiaConejo; }
    public double getModEnergiaLobo() { return modEnergiaLobo; }
    public double getModExitoCazaLobo() { return modExitoCazaLobo; }
}
```

2. Aclaración de las funciones que generan dudas

Aquí tienes el detalle exacto de cómo deben funcionar las clases e interfaces que el enunciado deja abiertas o ambiguas:

A. Clase Entidad y método `envejecer()`

- `envejecer()`: Incrementa `edad++` y resta un valor base de energía por turno (por ejemplo, `energia -= 2`). Luego llama a `verificarMuerte()` si implementa `Mortal` o directamente ajusta la energía con el setter para no bajar de 0.
- `actuar(Ecosistema eco)`: Método abstracto. En cada subclase ejecutará la lógica propia del turno.

B. Interface `Mortal`

- `estaVivo()`: Retorna `energia > 0` (o valida la variable `viva`).
- `morir()`: Cambia `viva = false` e imprime un mensaje en consola (ej. *"Conejo 'Rex' ha muerto"*).
- `verificarMuerte()` (método default):

```
default void verificarMuerte() {
    if (!estaVivo()) {
        morir();
    }
}
```

C. Interface `Reproducible`

- `puedeReproducirse()`: Evalúa la condición. Por ejemplo, en `Conejo`: `energia > 60` y que exista otro conejo vivo. En `Planta`: `energia > 40` y `clima != Clima.INVIERNO`.
- `reproducirse(Ecosistema eco)`: Crea un nuevo objeto de su misma especie con energía inicial y lo agrega al ecosistema.
- `intentarReproduccion(Ecosistema eco)` (método default):

```
default void intentarReproduccion(Ecosistema eco) {
    if (puedeReproducirse()) {
        reproducirse(eco);
    }
}
```

D. Clase `Planta`

- `serComida()`: Retorna el valor nutritivo (`tamano * 10`) y reduce su energía al mínimo (o la marca como muerta para consumirla por completo).
- `actuar(Ecosistema eco)`: Llama a `intentarReproduccion(eco)` respetando los efectos del clima actual.

E. Clase `Conejo`

- **actuar(Ecosistema eco):** Llama a `comer(eco)` y después a `intentarReproduccion(eco)`.
- **comer(Ecosistema eco):**
 1. Busca en `eco.getPlantas()` una planta viva.
 2. Si encuentra una: llama a `planta.serComida()`, suma esa energía a su propia energía y remueve o mata la planta.
 3. Si no encuentra ninguna: se resta 15 de energía.

F. Clase Lobo

- **actuar(Ecosistema eco):** Llama a `comer(eco)`.
- **comer(Ecosistema eco):**
 1. Busca un conejo vivo en `eco.getConejos()`.
 2. Calcula la probabilidad de éxito de la caza basada en la energía actual del lobo multiplicada por el factor del clima (por ejemplo, $\text{probabilidad} = (\text{energia} / 100.0) * \text{climaActual.getModExitoCazaLobo}()$).
 3. Genera un número aleatorio `Math.random()`. Si entra dentro del margen de éxito: mata al conejo (`conejo.morir()`), gana energía, e incrementa `exitosCaza++`. Si falla, no gana energía.

G. Requisito de Sobrecarga

El enunciado pide obligatoriamente aplicar **Sobrecarga**. Puedes resolverlo fácilmente en `Ecosistema`:

```
// Versión 1: Crea entidad con energía por defecto
public void agregarEntidad(String tipo) { ... }
```

```
// Versión 2 (Sobrecargada): Crea entidad permitiendo definir la energía inicial
public void agregarEntidad(String tipo, double energiaInicial) { ... }
```

3. Dinámica de ejecución del Ecosistema (procesarTurno)

En cada turno del bucle principal, el método `procesarTurno()` debe ejecutar las acciones en el siguiente orden estricto:

1. **Efectos climáticos del turno:** Aplicar pérdidas o ganancias automáticas de energía según el clima activo.
2. **Acción de Entidades:** Iterar sobre plantas, conejos y lobos ejecutando `actuar(this)`.
3. **Envejecimiento:** Llamar a `envejecer()` en todas las entidades vivas.
4. **Limpieza:** Remover de las listas `ArrayList` a aquellas entidades cuya energía sea 0 o `viva == false`.
5. **Comprobación:** Verificar mediante `ecosistemaColapsado()` si alguna lista quedó vacía.

¿Te gustaría que desarrollemos el código base de alguna de estas clases en particular o la estructura del menú principal para ayudarte a arrancar?

User prompt: me gustaria ir revisando clase por clase, aqui te mando la clase animal package test.parcial1; abstract class Animal extends Entidad implements Mortal{ //atributos private int velocidad; private double peso; //metodos abstract void comer(Ecosistema eco); private void moverse(){ System.out.println("El animal se movio"); }}
tambien me devuelve el siguiente error: constructor Entidad in class Entidad cannot be applied to given types; required: String, double, int, boolean; found: no arguments; reason: actual and formal arguments lists differ in lenght

Response: ¡Excelente enfoque! Vamos a revisar la clase `Animal` paso a paso para resolver tanto el error que te muestra el compilador como algunos detalles de diseño solicitados en la consigna.

1. Causa del error de compilación

El error:

```
constructor Entidad in class Entidad cannot be applied to given types; required: String, double, int, boolean; found: no arguments...
```

Ocurre porque en Java, cuando una subclase no define un constructor explícito, el compilador inserta de forma automática un constructor por defecto sin parámetros: `public Animal() { super(); }`.

Sin embargo, como la clase padre `Entidad` tiene un constructor personalizado con parámetros (`String nombre`, `double energia`, `int edad`, `boolean viva`), Java necesita que la clase hija llame explícitamente a `super(...)` pasando esos datos.

2. Observaciones en el código de `Animal`

1. Visibilidad del método `moverse()`:

- En tu código está marcado como `private`:

```
private void moverse() { ... }
```

- Si es `private`, las clases hijas (`Conejo` y `Lobo`) no podrán heredarlo ni utilizarlo. Debe ser `public` o `protected` para que los animales concretos puedan llamar a `moverse()`.

2. Visibilidad de la clase y métodos:

- Es recomendable agregar el modificador `public` tanto a la clase como a sus métodos abstractos para asegurar un correcto encapsulamiento entre paquetes.

3. Código corregido para `Animal.java`

Aquí tienes la versión corregida incorporando el constructor obligatorio que delega los atributos a `Entidad` mediante `super(...)`:

```
package test.parcial1;

public abstract class Animal extends Entidad implements Mortal {
    // Atributos adicionales especificos de Animal
```

```

private int velocidad;
private double peso;

// Constructor que invoca al constructor de Entidad (resuelve el error de compilacion)
public Animal(String nombre, double energia, int edad, boolean viva, int velocidad, double peso) {
    super(nombre, energia, edad, viva);
    this.velocidad = velocidad;
    this.peso = peso;
}

// Metodo abstracto a implementar por Conejo y Lobo
public abstract void comer(Ecosistema eco);

// Metodo concreto compartido (cambiado de private a public/protected para que las subclases puedan usarlo)
public void moverse() {
    System.out.println(getNombre() + " se ha desplazado.");
}

// Getters y Setters
public int getVelocidad() {
    return velocidad;
}

public void setVelocidad(int velocidad) {
    this.velocidad = velocidad;
}

public double getPeso() {
    return peso;
}

public void setPeso(double peso) {
    this.peso = peso;
}
}

```

Sugerencia para la siguiente revisión

Para que todo compile de forma coordinada, nos conviene revisar la clase base **Entidad** o la interfaz **Mortal**. ¿Te gustaría que analicemos el código de Entidad a continuación?

User prompt: genial, ahora te paso la clase Conejo: package test.parcial1; public class Conejo extends Animal implements Reproducible{ //metodos public void comer(){ //busca una planta viva en el ecosistema y la come; si no hay, pierde 15 de energía } public void actuar(){ comer(); reproducirse(eco); } @Override public void mostrarEstado(){ System.out.println(this.getNombre()); System.out.println(this.getEnergia()); if (this.getEnergia() < 20){ System.out.println("Peligro!!"); } }

public void reproduccion(){ // si energía > 60 y hay al menos otro conejo vivo, puede generar un nuevo Conejo } } en este caso el error que me devuelve es el siguiente:
Conejo is not abstract and does not override abstract method comer(Ecosistema) in animal

Response: Vamos a analizar los errores de compilación de la clase Conejo y a completar la lógica que te pide la consigna.

1. Errores de compilación en tu código

1. Conejo is not abstract and does not override abstract method comer(Ecosistema) in Animal:

- Ocurre porque en Animal definiste `abstract void comer(Ecosistema eco)` (recibe un parámetro Ecosistema). En Conejo escribiste `public void comer()` (sin parámetros). Al cambiar los parámetros, Java interpreta que es otro método distinto y que no estás implementando el método abstracto obligatorio.

2. actuar() no coincide con el método abstracto de Entidad:

- En Entidad, el método es `actuar(Ecosistema eco)`. Debe recibir la instancia de Ecosistema para pasársela a `comer(eco)` y a `intentarReproduccion(eco)`.

3. Falta del constructor:

- Al heredar de Animal (que a su vez hereda de Entidad), Conejo necesita definir un constructor que llame a `super(...)`.

4. Nombres de métodos de la interfaz Reproducible:

- La interfaz requiere implementar `reproducirse(Ecosistema eco)` y `puedeReproducirse()`. En `actuar()`, se recomienda llamar al método default `intentarReproduccion(eco)` para aplicar el patrón de diseño pedido en el enunciado.

5. Implementación de Mortal:

- Como Conejo extiende de Animal, e interface Mortal define `estaVivo()` y `morir()`, debemos implementar esos dos métodos o delegarlos correctamente.

2. Código corregido e implementado para Conejo.java

```
package test.parcial1;

import java.util.ArrayList;

public class Conejo extends Animal implements Reproducible {

    // Constructor que traslada la informacion a la clase padre Animal
    public Conejo(String nombre, double energia, int edad, boolean viva, int velocidad, double peso) {
        super(nombre, energia, edad, viva, velocidad, peso);
    }

    // Constructor sobrecargado util para cuando nace un nuevo conejo durante la simulacion
    public Conejo(String nombre) {
        super(nombre, 50.0, 0, true, 10, 2.5); // Valores iniciales por defecto
    }
}
```

```

@Override
public void actuar(Ecosistema eco) {
    if (!isViva()) return;

    comer(eco);
    // Usa el metodo default de la interfaz Reproducible
    intentarReproduccion(eco);
}

@Override
public void comer(Ecosistema eco) {
    ArrayList<Planta> plantas = eco.getPlantas();
    Planta plantaEncontrada = null;

    // Busca la primera planta viva disponible en el ecosistema
    for (Planta p : plantas) {
        if (p.isViva()) {
            plantaEncontrada = p;
            break;
        }
    }

    if (plantaEncontrada != null) {
        double energiaGanada = plantaEncontrada.serComida();
        setEnergia(getEnergia() + energiaGanada);
        System.out.println("Conejo " + getNombre() + " comio " + plantaEncontrada.getNombre() + " (" + energiaGanada + " energia)");
    } else {
        // Si no encuentra comida, pierde 15 de energia
        setEnergia(getEnergia() - 15);
        System.out.println("Conejo " + getNombre() + " no encontro comida (-15 energia)");
    }
}

@Override
public boolean puedeReproducirse() {
    // Requiere energia > 60
    return isViva() && getEnergia() > 60;
}

@Override
public void reproducirse(Ecosistema eco) {
    // Verifica si hay al menos otro conejo vivo en el ecosistema
    long conejosVivos = eco.getConejos().stream().filter(Entidad::isViva).count();

    if (conejosVivos >= 2) {
        String nombreNuevo = "Conejo-" + (eco.getConejos().size() + 1);
    }
}

```

```

        Conejo nuevoConejo = new Conejo(nombreNuevo);
        eco.getConejos().add(nuevoConejo);

        // Procrear consume algo de energia de la madre
        setEnergia(getEnergia() - 20);
        System.out.println("Conejo '" + getNombre() + "' se reprodujo. Nacio '" + nombreNuevo + "'");
    }
}

@Override
public void mostrarEstado() {
    System.out.print("Conejo: " + getNombre() + " | Energia: " + getEnergia());
    if (getEnergia() < 20) {
        System.out.print(" [PELIGRO: energia=" + getEnergia() + "]");
    }
    System.out.println();
}

// Métodos obligatorios requeridos por la interfaz Mortal
@Override
public boolean estaVivo() {
    return isViva() && getEnergia() > 0;
}

@Override
public void morir() {
    setViva(false);
    setEnergia(0);
    System.out.println("Conejo '" + getNombre() + "' ha muerto.");
}
}

```

¿Cómo lo ves? Si quieres, podemos pasar a revisar la clase **Lobo**, la interfaz **Reproducible** o la interfaz **Mortal**.

User prompt: aqui esta la clase ecosistema:

Response: ¡Hola! Parece que el mensaje con el código de la clase Ecosistema no se llegó a enviar o quedó incompleto.

Por favor, vuelve a pegar la clase **Ecosistema** para que la revisemos paso a paso, corrijamos los errores que tenga y la adaptemos a las consignas del trabajo.

User prompt: package test.parcial1; import java.util.ArrayList; public class Ecosistema { //atributos private ArrayList<Planta> plantas; private ArrayList<Conejo> conejos; private ArrayList<Lobo> lobos; //private Clima climaActual; private int turnoActual; }

Response: ¡Genial! Vamos a construir la clase Ecosistema completa. Como habíamos visto, el tema del clima lo resolvemos con el enum `Clima` que creamos anteriormente.

En esta clase debemos incluir:

1. **Atributos y Constructor** para inicializar todas las listas.
2. **Métodos principales:** `procesarTurno()`, `mostrarEstado()`, `ecosistemaColapsado()` y `generarReporteFinal()`.
3. **Sobrecarga de métodos** en `agregarEntidad()` (requisito obligatorio de POO de la consigna).
4. **Control de límites** (por ejemplo, validar no agregar más de 5 lobos).

Código completo e implementado de `Ecosistema.java`

```
package test.parcial1;

import java.util.ArrayList;
import java.util.Iterator;

public class Ecosistema {
    // Atributos obligatorios
    private ArrayList<Planta> plantas;
    private ArrayList<Conejo> conejos;
    private ArrayList<Lobo> lobos;
    private Clima climaActual;
    private int turnoActual;

    // Métricas para el reporte final
    private int totalNacimientosPlantas = 0;
    private int totalNacimientosConejos = 0;
    private int totalMuertesPlantas = 0;
    private int totalMuertesConejos = 0;
    private int totalMuertesLobos = 0;
    private int lobosAgregadosEnIntervencion = 0;

    // Constructor
    public Ecosistema(Clima climaInicial) {
        this.plantas = new ArrayList<>();
        this.conejos = new ArrayList<>();
        this.lobos = new ArrayList<>();
        this.climaActual = climaInicial;
        this.turnoActual = 1;
    }

    // =====
    // REQUISITO POO: SOBRECARGA DE MÉTODOS (agregarEntidad)
```

```
// =====

// Versión 1: Agrega entidad con valores/energía por defecto
public boolean agregarEntidad(String tipo) {
    return agregarEntidad(tipo, -1); // Usa -1 para indicar valor por defecto
}

// Versión 2 (Sobrecargada): Agrega entidad permitiendo definir energía inicial
public boolean agregarEntidad(String tipo, double energiaInicial) {
    tipo = tipo.toLowerCase().trim();

    switch (tipo) {
        case "planta":
            double energiaP = (energiaInicial > 0) ? energiaInicial : 30.0;
            int tamaño = (int) (Math.random() * 5) + 1; // Tamaño de 1 a 5
            Planta nuevaPlanta = new Planta("Planta-" + (plantas.size() + 1), energiaP, 0, true, tamaño);
            plantas.add(nuevaPlanta);
            totalNacimientosPlantas++;
            System.out.println("Se agrego 'Planta-' + plantas.size() + '' al ecosistema.");
            return true;

        case "conejo":
            double energiaC = (energiaInicial > 0) ? energiaInicial : 50.0;
            Conejo nuevoConejo = new Conejo("Conejo-" + (conejos.size() + 1), energiaC, 0, true, 10, 2.5);
            conejos.add(nuevoConejo);
            totalNacimientosConejos++;
            System.out.println("Se agrego 'Conejo-' + conejos.size() + '' al ecosistema.");
            return true;

        case "lobo":
            // Validación: Máximo 5 lobos en toda la simulación
            if (lobos.size() >= 5) {
                System.out.println("No se pueden agregar mas lobos. Limite maximo alcanzado (5 lobos).");
                return false;
            }
            double energiaL = (energiaInicial > 0) ? energiaInicial : 70.0;
            Lobo nuevoLobo = new Lobo("Lobo-" + (lobos.size() + 1), energiaL, 0, true, 15, 20.0);
            lobos.add(nuevoLobo);
            lobosAgregadosEnIntervencion++;
            System.out.println("Se agrego 'Lobo-' + lobos.size() + '' al ecosistema.");
            return true;

        default:
            System.out.println("Tipo de entidad no reconocido: " + tipo);
            return false;
    }
}
```

```
}

// =====
// LÓGICA DE SIMULACIÓN POR TURNO
// =====

public void procesarTurno() {
    System.out.println("\n=== TURNO " + turnoActual + " Clima: " + climaActual + " ===");
    mostrarConteo();
    System.out.println("-- Eventos --");

    // 1. Aplicar efectos del clima en la energía base
    aplicarEfectosClima();

    // 2. Actuar: Plantas intentan reproducirse
    for (int i = 0; i < plantas.size(); i++) {
        Planta p = plantas.get(i);
        if (p.isViva()) {
            p.actuar(this);
        }
    }

    // 3. Actuar: Conejos comen y se reproducen
    for (int i = 0; i < conejos.size(); i++) {
        Conejo c = conejos.get(i);
        if (c.isViva()) {
            c.actuar(this);
        }
    }

    // 4. Actuar: Lobos cazan
    for (int i = 0; i < lobos.size(); i++) {
        Lobo l = lobos.get(i);
        if (l.isViva()) {
            l.actuar(this);
        }
    }

    // 5. Envejecimiento y gasto de energía base
    envejecerEntidades();

    // 6. Limpiar entidades muertas del ecosistema
    limpiarMuertos();

    // 7. Mostrar resumen final del turno
    mostrarEstado();
}
```

```
        turnoActual++;
    }

    private void aplicarEfectosClima() {
        // Conejos
        for (Conejo c : conejos) {
            if (c.isViva()) {
                c.setEnergia(c.getEnergia() + climaActual.getModEnergiaConejo());
            }
        }
        // Lobos
        for (Lobo l : lobos) {
            if (l.isViva()) {
                l.setEnergia(l.getEnergia() + climaActual.getModEnergiaLobo());
            }
        }
    }

    private void envejecerEntidades() {
        for (Planta p : plantas) if (p.isViva()) p.envejecer();
        for (Conejo c : conejos) if (c.isViva()) c.envejecer();
        for (Lobo l : lobos) if (l.isViva()) l.envejecer();
    }

    private void limpiarMuertos() {
        // Remover plantas muertas
        Iterator<Planta> itP = plantas.iterator();
        while (itP.hasNext()) {
            Planta p = itP.next();
            if (!p.estaVivo()) {
                totalMuertesPlantas++;
                itP.remove();
            }
        }

        // Remover conejos muertos
        Iterator<Conejo> itC = conejos.iterator();
        while (itC.hasNext()) {
            Conejo c = itC.next();
            if (!c.estaVivo()) {
                totalMuertesConejos++;
                itC.remove();
            }
        }

        // Remover lobos muertos
    }
```

```

        Iterator<Lobo> itL = lobos.iterator();
        while (itL.hasNext()) {
            Lobo l = itL.next();
            if (!l.estaVivo()) {
                totalMuertesLobos++;
                itL.remove();
            }
        }
    }

    public boolean ecosistemaColapsado() {
        return plantas.isEmpty() || conejos.isEmpty() || lobos.isEmpty();
    }

    public void cambiarClima(Clima nuevoClima) {
        this.climaActual = nuevoClima;
        System.out.println("El clima ha cambiado a: " + nuevoClima);
    }

    public void mostrarConteo() {
        System.out.println("Plantas: " + plantas.size() + " | Conejos: " + conejos.size() + " | Lobos: " + lobos.size());
    }

    public void mostrarEstado() {
        System.out.print("Estado: ");
        mostrarConteo();
    }

    public void generarReporteFinal() {
        System.out.println("\n=====");
        System.out.println("                REPORTE FINAL DE SIMULACIÓN                ");
        System.out.println("=====");

        if (ecosistemaColapsado()) {
            System.out.println("Causa de fin: COLAPSO DEL ECOSISTEMA");
            if (plantas.isEmpty()) System.out.println("-> Se extinguieron las PLANTAS.");
            if (conejos.isEmpty()) System.out.println("-> Se extinguieron los CONEJOS.");
            if (lobos.isEmpty()) System.out.println("-> Se extinguieron los LOBOS.");
        } else {
            System.out.println("Causa de fin: Se alcanzaron los turnos maximos configurados.");
        }

        System.out.println("\n--- Estadisticas Generales ---");
        System.out.println("Turnos jugados: " + (turnoActual - 1));
        System.out.println("Nacimientos Totales: Plantas=" + totalNacimientosPlantas + ", Conejos=" + totalNacimientosConejos);
        System.out.println("Muertes Totales: Plantas=" + totalMuertesPlantas + ", Conejos=" + totalMuertesConejos + ", Lobos=" + totalMuertesLobos);
    }

```

```

// Buscar lobo con mas cacerias exitosas
Lobo mejorCazador = null;
for (Lobo l : lobos) {
    if (mejorCazador == null || l.getExitosCaza() > mejorCazador.getExitosCaza()) {
        mejorCazador = l;
    }
}
if (mejorCazador != null) {
    System.out.println("Lobo con mas cacerias exitosas: " + mejorCazador.getNombre() + " (" + mejorCazador.getExitosCaza() + " cacerias)");
}

System.out.println("=====\n");
}

// =====
// GETTERS Y SETTERS
// =====

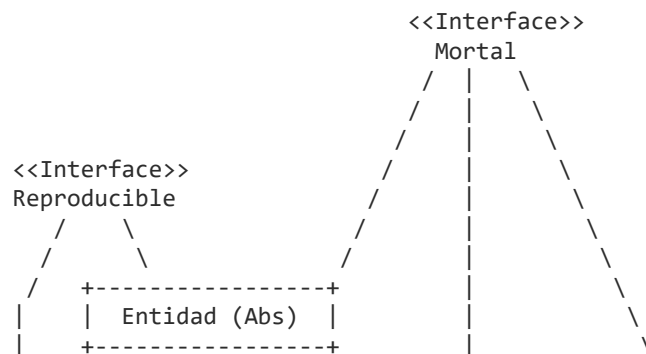
public ArrayList<Planta> getPlantas() { return plantas; }
public ArrayList<Conejo> getConejos() { return conejos; }
public ArrayList<Lobo> getLobos() { return lobos; }
public Clima getClimaActual() { return climaActual; }
public int getTurnoActual() { return turnoActual; }
}

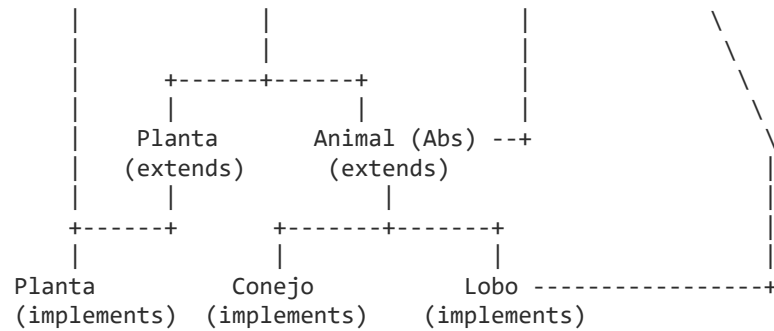
```

¿Qué te parece? Si quieres, podemos revisar ahora la clase **Lobo**, la interfaz **Mortal** o la interfaz **Reproducible** para dejar todo el modelo compilando perfectamente.

User prompt: antes de continuar, me podrias mostrar un arbol genealogico de las clases e interfaces, asi puedo ver bien cuales son las que heredan de cuales

Response: Aquí tienes la jerarquía completa de clases e interfaces descritas en las consignas del proyecto:





Resumen de relaciones por entidad

1. Entidad (Clase Abstracta)

- **Es la clase raíz** de todas las entidades del ecosistema.
- Define atributos comunes: nombre, energia, edad, viva.

2. Animal (Clase Abstracta)

- **Hereda de:** Entidad (extends Entidad).
- **Implementa:** Mortal (implements Mortal).
- Es una capa intermedia con atributos de movilidad: velocidad, peso.

3. Planta (Clase Concreta)

- **Hereda de:** Entidad (extends Entidad).
- **Implementa:** Reproducible (implements Reproducible).
- *(Opcional)* Puede implementar también Mortal.

4. Conejo (Clase Concreta)

- **Hereda de:** Animal (extends Animal).
- **Implementa:** Reproducible (implements Reproducible) y Mortal (heredada de Animal).

5. Lobo (Clase Concreta)

- **Hereda de:** Animal (extends Animal).
- **Implementa:** Mortal (heredada de Animal).

- **NO implementa** Reproducible (los lobos no se reproducen en esta simulación).

Mapeo de Interfaces

- **Reproducible:** Métodos reproducirse(), puedeReproducirse(), y el método default intentarReproduccion(). La implementan **Planta** y **Conejo**.
- **Mortal:** Métodos estaVivo(), morir(), y el método default verificarMuerte(). La implementa **Animal** (y por extensión Conejo y Lobo).

Cuando quieras, avanzamos con la clase **Lobo**, **Planta** o con las **Interfaces**.

User prompt: tambien una cosita que me molesta en netbeans, capaz podes ayudarme. En la foto que te mande me gustaria saber como sacar la linea violeta que esta en la mitad de la pantalla

Response: Esa línea vertical violeta en NetBeans se conoce como **Right Margin Guide** (guía del margen derecho). Su función es indicar hasta qué columna (generalmente la columna 80) conviene escribir código para evitar líneas de texto demasiado largas.

Para ocultarla o desactivarla en NetBeans, sigue estos pasos:

1. Ve al menú superior y selecciona **Tools** (Herramientas) > **Options** (Opciones). (En macOS: **NetBeans** > **Preferences**)
2. Selecciona el ícono de **Editor** en la parte superior.
3. Ve a la pestaña **Formatting** (Formato).
4. Busca la opción **Right Margin** (Margen derecho).
5. Desmarca la casilla que dice **Show Right Margin Line** (Mostrar línea de margen derecho) o cambia el valor del margen si prefieres moverla.
6. Haz clic en **Apply** y luego en **OK**.

La línea desaparecerá de la pantalla.

User prompt: no veo que me deje sacar la opcion de right margin

Response: En la versión de NetBeans que tienes abierta, esa opción está ubicada en otra pestaña dentro de **Editor**.

Para ocultar la línea, sigue estos dos pasos sencillos:

1. En esa misma ventana de opciones, haz clic en la pestaña **Formatting** en la que estás actualmente y pon el número de **Right Margin** en un valor muy alto (por ejemplo, **500**). Esto alejará la línea tanto a la derecha que desaparecerá completamente de tu vista.
2. Si prefieres desactivar el renderizado del margen por completo:
 - Haz clic en la pestaña **General** (está dos pestañas a la izquierda de **Formatting**, en la segunda fila superior de pestañas).

- Desmarca la casilla **Show Right Margin** (o *Text Limit Line*).
 - Haz clic en **Apply** y **OK**.
-

User prompt: acabo de poner el valor en 500 y no se guardo ningun cambio a pesar de apply y ok

Response: Ese comportamiento ocurre porque cuando la casilla **"Use All Languages Settings"** (arriba a la izquierda en tu captura) está desmarcada, el valor de *Right Margin* aplica únicamente según el lenguaje seleccionado o hereda la configuración global.

Para solucionarlo directamente y hacer desaparecer la línea de inmediato:

1. Vuelve a **Tools > Options > Editor**.
2. Ve a la pestaña **Appearance** (Aparencia) en las pestañas principales superiores (está entre *Team* y *Miscellaneous*).
3. Entra a la pestaña interna **Document Line Limit** (o *Editor Settings* según la versión).
4. Desmarca la casilla **Show Right Margin Line** (o **Show Text Limit Line**).
5. Da clic en **Apply** y luego en **OK**.

Alternativa rápida: Si estás en la pestaña **Formatting** donde sacaste la foto:

- Marca la casilla **Use All Languages Settings**.
 - Pon **Right Margin** en 500.
 - Haz clic en **Apply** y **OK**.
-

User prompt: tampoco me aparece

Response: En las versiones más recientes de Apache NetBeans cambiaron la ubicación exacta de esa opción.

Para desactivar la línea por completo de forma definitiva, sigue estos pasos:

1. Ve a la pestaña **Editor** (el ícono del lápiz, arriba a la izquierda).
2. Haz clic en la subpestaña **Formatting** (Formato).
3. Asegúrate de que el desplegable **Language** diga **All Languages** (o activa la casilla **Use All Languages Settings** que está justo abajo).
4. Cambia el campo **Right Margin** a un número muy grande como 999.
5. Haz clic en **Apply** y luego en **OK**.

¿Por qué no te funcionó antes?

En tu foto anterior tenías seleccionado Language: Java. Si lo cambias solo en Java pero la regla global o del archivo abierto está heredando de otro perfil, no aplica el cambio visual en la ventana actual.

Al cambiar el **Language** a **All Languages** (o marcar la casilla *Use All Languages Settings*), forzarás a que la línea se mueva a la columna 999 en todos los archivos y desaparecerá de tu vista de inmediato.

User prompt: ahi esta, no se porque lo puse en 500 otra vez mas y esta vez lo hizo jajaja

Response: ¡Jajaja, excelente! A veces NetBeans se pone un poco caprichoso y necesita que le confirmes el cambio dos veces o que reenfoque la pestaña para refrescar el renderizado de la pantalla. Lo importante es que ya te libraste de esa línea tan molesta.

Cuando quieras, continuamos revisando las clases que nos faltan: **Lobo**, **Planta**, las interfaces (**Mortal**, **Reproducible**) o la clase **Main** con la lógica del juego. ¿Por cuál te gustaría seguir?

User prompt: ahi con mi grupo nos organizamos que tiene que hacer c/u, te mando lo que me toca hacer a mi MATI — Modelo base + sistema de mortalidad + estadísticas Archivos principales Entidad.java Animal.java Mortal.java Estadísticas.java (nuevo, si resulta necesario) 1. Entidad.java Completar la clase abstracta base. Debe contener como atributos privados: nombre : String energia : double edad : int viva : boolean Implementar constructor con parámetros. Implementar getters y setters. El setter de energía debe impedir valores negativos: si energia < 0 → energia = 0 Definir: actuar(Ecosistema eco) mostrarEstado() como métodos abstractos. Implementar: envejecer() que debe aumentar la edad y descontar la energía base correspondiente. 2. Animal.java Debe: extends Entidad implements Mortal Agregar atributos privados: velocidad : int peso : double Crear constructor, getters y setters. Definir: comer(Ecosistema eco) como método abstracto. Implementar: moverse() que informa que el animal se desplazó. 3. Mortal.java Implementar: estaVivo() morir() y el método default: verificarMuerte() Si la energía llega a 0, debe provocar la muerte e informar el evento. Estos elementos son obligatorios según la estructura de POO definida por la profesora. 1InstanciaEvaluativa-InterfazGr... 1InstanciaEvaluativa-InterfazGr... 4. BONUS — Estadísticas en tiempo real Mati también se encarga del bonus de estadísticas. Registrar por cada turno: cantidad de plantas cantidad de conejos cantidad de lobos Al terminar permitir determinar: turno de máximo de plantas turno de mínimo de plantas turno de máximo de conejos turno de mínimo de conejos turno de máximo de lobos turno de mínimo de lobos Es el bonus de historial de poblaciones solicitado por la consigna.

Response: ¡Excelente división de tareas! Tu parte es fundamental porque estás construyendo los **cimientos del proyecto** (la clase base Entidad, la capa Animal, el contrato de mortalidad Mortal y el módulo extra de Estadísticas para los puntos bonus).

A continuación te paso el código exacto y completo para cada uno de los 4 archivos de tu responsabilidad, listo para copiar y pegar en NetBeans.

1. Entidad.java (Clase Abstracta Base)

```
package test.parcial1;

public abstract class Entidad {
    // Atributos privados requeridos
    private String nombre;
    private double energia;
    private int edad;
    private boolean viva;
```

```
// Constructor con parametros
public Entidad(String nombre, double energia, int edad, boolean viva) {
    this.nombre = nombre;
    setEnergia(energia); // Usa el setter para aplicar la validacion desde el inicio
    this.edad = edad;
    this.viva = viva;
}

// Métodos abstractos requeridos
public abstract void actuar(Ecosistema eco);
public abstract void mostrarEstado();

// Método concreto: incrementa edad y descuenta energía base por existir
public void envejecer() {
    this.edad++;
    setEnergia(this.energia - 2.0); // Descuento de energía base por turno
}

// Getters y Setters con validación en la energía
public String getNombre() {
    return nombre;
}

public void setNombre(String nombre) {
    this.nombre = nombre;
}

public double getEnergia() {
    return energia;
}

// Validación obligatoria: si energía < 0 se lleva a 0
public void setEnergia(double energia) {
    if (energia < 0) {
        this.energia = 0;
    } else {
        this.energia = energia;
    }
}

public int getEdad() {
    return edad;
}

public void setEdad(int edad) {
    this.edad = edad;
}
```

```
    }

    public boolean isViva() {
        return viva;
    }

    public void setViva(boolean viva) {
        this.viva = viva;
    }
}
```

2. Mortal.java (Interfaz)

```
package test.parcial1;

public interface Mortal {
    // Métodos abstractos a implementar
    boolean estaVivo();
    void morir();

    // Método default: evalúa si la energía llegó a 0 para provocar la muerte
    default void verificarMuerte() {
        if (!estaVivo()) {
            morir();
        }
    }
}
```

3. Animal.java (Clase Abstracta Intermedia)

```
package test.parcial1;

public abstract class Animal extends Entidad implements Mortal {
    // Atributos privados adicionales
    private int velocidad;
    private double peso;

    // Constructor que invoca al constructor de Entidad
    public Animal(String nombre, double energia, int edad, boolean viva, int velocidad, double peso) {
        super(nombre, energia, edad, viva);
        this.velocidad = velocidad;
        this.peso = peso;
    }

    // Método abstracto obligatorio para las subclases (Conejo, Lobo)
```

```

    public abstract void comer(Ecosistema eco);

    // Método concreto compartido
    public void moverse() {
        System.out.println("El animal '" + getNombre() + "' se desplazo.");
    }

    // Implementación de la interfaz Mortal
    @Override
    public boolean estaVivo() {
        return isViva() && getEnergia() > 0;
    }

    @Override
    public void morir() {
        setViva(false);
        setEnergia(0);
        System.out.println("El animal '" + getNombre() + "' ha muerto.");
    }

    // Getters y Setters
    public int getVelocidad() {
        return velocidad;
    }

    public void setVelocidad(int velocidad) {
        this.velocidad = velocidad;
    }

    public double getPeso() {
        return peso;
    }

    public void setPeso(double peso) {
        this.peso = peso;
    }
}

```

4. Estadisticas.java (BONUS — Estadísticas en tiempo real)

Esta clase registra el conteo de plantas, conejos y lobos en cada turno y, al finalizar la simulación, calcula exactamente en qué turno cada población alcanzó sus máximos y mínimos.

```

package test.parcial1;

import java.util.ArrayList;

```

```
public class Estadisticas {

    // Clase interna auxiliar para guardar la foto de cada turno
    private static class RegistroTurno {
        int turno;
        int plantas;
        int conejos;
        int lobos;

        RegistroTurno(int turno, int plantas, int conejos, int lobos) {
            this.turno = turno;
            this.plantas = plantas;
            this.conejos = conejos;
            this.lobos = lobos;
        }
    }

    private ArrayList<RegistroTurno> historial;

    public Estadisticas() {
        this.historial = new ArrayList<>();
    }

    // Registra la población al finalizar cada turno
    public void registrarTurno(int turno, int plantas, int conejos, int lobos) {
        historial.add(new RegistroTurno(turno, plantas, conejos, lobos));
    }

    // Procesa e imprime el informe de máximos y mínimos del Bonus
    public void mostrarReporteEstadistico() {
        if (historial.isEmpty()) {
            System.out.println("No hay datos estadisticos registrados.");
            return;
        }

        RegistroTurno maxP = historial.get(0), minP = historial.get(0);
        RegistroTurno maxC = historial.get(0), minC = historial.get(0);
        RegistroTurno maxL = historial.get(0), minL = historial.get(0);

        for (RegistroTurno r : historial) {
            // Plantas
            if (r.plantas > maxP.plantas) maxP = r;
            if (r.plantas < minP.plantas) minP = r;

            // Conejos
            if (r.conejos > maxC.conejos) maxC = r;
```

```

        if (r.conejos < minC.conejos) minC = r;

        // Lobos
        if (r.lobos > maxL.lobos) maxL = r;
        if (r.lobos < minL.lobos) minL = r;
    }

    System.out.println("\n=== ESTADÍSTICAS EN TIEMPO REAL (HISTORIAL POPULACIONAL) ===");
    System.out.println("PLANTAS:");
    System.out.println("  - Maximo: " + maxP.plantas + " en el Turno " + maxP.turno);
    System.out.println("  - Minimo: " + minP.plantas + " en el Turno " + minP.turno);

    System.out.println("CONEJOS:");
    System.out.println("  - Maximo: " + maxC.conejos + " en el Turno " + maxC.turno);
    System.out.println("  - Minimo: " + minC.conejos + " en el Turno " + minC.turno);

    System.out.println("LOBOS:");
    System.out.println("  - Maximo: " + maxL.lobos + " en el Turno " + maxL.turno);
    System.out.println("  - Minimo: " + minL.lobos + " en el Turno " + minL.turno);
    System.out.println("=====");
}
}

```

¿Cómo conecta tu parte con la de tus compañeros?

- Quien haga Ecosistema debe tener un atributo `private Estadisticas estadisticas;` y llamar a `estadisticas.registrarTurno(...)` al final de cada turno. Al terminar la simulación, llamará a `estadisticas.mostrarReporteEstadistico()`.
- Quienes hagan Planta, Conejo o Lobo heredarán de tus clases (Entidad o Animal) e implementará `actuar(eco)` y `comer(eco)` sobre la base que dejaste definida.

User prompt: ahí te voy mandando lo que tengo hecho y lo completamos

Response: ¡Perfecto! Estoy listo. Ve pasándome los archivos o Fragmentos de código que tengas armados uno por uno y los revisamos juntos para completarlos, corregir cualquier detalle y asegurarnos de que cumplan con todos los requisitos de POO de la consigna.

```

User prompt: Entidad.java: package ecosim; abstract class Entidad { //ATRIBUTOS  private String nombre;  private double energia;  private int edad;  private boolean viva;
//CONSTRUCTOR  public Entidad(String nombre, double energia, int edad, boolean viva){  this.nombre = nombre;  this.energia = energia;  this.edad = edad;
this.viva = viva;  } //GETS  public String getNombre(){  return this.nombre;  }  public double getEnergia(){  return this.energia;  }  public int getEdad(){
return this.edad;  }  public boolean getViva(){  return this.viva;  } //SETS  public void setNombre(String nombre){  this.nombre = nombre;  }  public void
setEnergia(double energia){  this.energia = energia;  }  public void setEdad(int edad){  this.edad = edad;  }  public void setViva(boolean viva){  this.viva =
viva;  } //METODOS  abstract void actuar(Ecosistema eco);  abstract void mostrarEstado();  public void envejecer(){  this.edad += 1;  this.energia -= 1;  }}

```

Response: Tu clase Entidad.java está muy bien encaminada, pero tiene tres detalles importantes que debes corregir para cumplir con las consignas de la profe y evitar problemas con el resto del grupo:

1. Visibilidad de la clase y métodos abstractos:

- Si las demás clases están en otros paquetes o si las interfaces necesitan acceder, conviene agregar `public` a la definición de la clase `public abstract class Entidad` y a sus métodos abstractos `public abstract void actuar(Ecosistema eco)`.

2. Validación en `setEnergia`:

- La consigna específica que la energía no puede ser negativa y debe ajustarse a 0 si baja de dicho valor. En el constructor también conviene llamar a `setEnergia(energia)` para aplicar esa misma regla desde el momento en que se crea la entidad.

3. Manejo de energía en `envejecer()`:

- Al restar energía en `envejecer()`, conviene usar `setEnergia(getEnergia() - 2.0)` (o la energía base que definan) para asegurarte de que si llega a valores bajo cero se lleve automáticamente a 0 mediante la validación del setter.

Versión corregida y completa de Entidad.java

```
package ecosim;

public abstract class Entidad {
    // ATRIBUTOS PRIVADOS
    private String nombre;
    private double energia;
    private int edad;
    private boolean viva;

    // CONSTRUCTOR
    public Entidad(String nombre, double energia, int edad, boolean viva) {
        this.nombre = nombre;
        setEnergia(energia); // Usa el setter con validacion desde el inicio
        this.edad = edad;
        this.viva = viva;
    }

    // METODOS ABSTRACTOS
    public abstract void actuar(Ecosistema eco);
    public abstract void mostrarEstado();

    // METODO CONCRETO: Envejecer
    public void envejecer() {
        this.edad += 1;
        setEnergia(this.energia - 2.0); // Resta energia base pasando por la validacion del setter
    }
}
```



```
// GETTERS
public String getNombre() {
    return this.nombre;
}

public double getEnergia() {
    return this.energia;
}

public int getEdad() {
    return this.edad;
}

public boolean getViva() { // 0 isViva()
    return this.viva;
}

// SETTERS CON VALIDACIÓN
public void setNombre(String nombre) {
    this.nombre = nombre;
}

// Validación obligatoria requerida por la consigna
public void setEnergia(double energia) {
    if (energia < 0) {
        this.energia = 0;
    } else {
        this.energia = energia;
    }
}

public void setEdad(int edad) {
    if (edad < 0) {
        this.edad = 0;
    } else {
        this.edad = edad;
    }
}

public void setViva(boolean viva) {
    this.viva = viva;
}
}
```

¡Pasame la siguiente clase o interfaz cuando quieras!

User prompt: package ecosim; abstract class Entidad { //ATRIBUTOS private String nombre; private double energia; private int edad; private boolean viva;
 //CONSTRUCTOR public Entidad(String nombre, double energia, int edad, boolean viva){ this.nombre = nombre; //validaciones en el constructor if (energia < 0){
 this.energia = 0; } else{ this.energia = energia; } if (edad < 0){ this.edad = 0; } else{ this.edad = edad; } this.viva = viva;
 } //GETS public String getNombre(){ return this.nombre; } public double getEnergia(){ return this.energia; } public int getEdad(){ return
 this.edad; } public boolean getViva(){ return this.viva; } //SETS public void setNombre(String nombre){ this.nombre = nombre; } public void
 setEnergia(double energia){ if (energia < 0){ this.energia = 0; } else{ this.energia = energia; } } public void setEdad(int edad){ if (edad <
 0){ this.edad = 0; } else{ this.edad = edad; } } public void setViva(boolean viva){ this.viva = viva; } //METODOS abstract void
 actuar(Ecosistema eco); abstract void mostrarEstado(); public void envejecer(){ this.edad += 1; setEnergia(this.energia - 2.0); //asi para que pase por la validacion
 }} asi quedaria bien entonces, puse la validacion en el constructor porque me tiraba un error

Response: ¡Quedó impecable! Esa es exactamente la forma adecuada de resolverlo.

Al poner la lógica de validación directamente o invocar a setEnergia y setEdad dentro del constructor, garantizas que ninguna entidad pueda instanciarse con valores inválidos desde el principio.

Solo quedan dos pequeños detalles recomendados antes de dar por cerrada esta clase:

1. **Visibilidad public:** Agrega public antes de abstract class Entidad y antes de los métodos abstractos public abstract void actuar(Ecosistema eco) y public abstract void mostrarEstado(). Si no se los pones, quedan con visibilidad *package-private* y te pueden dar problemas si implementas subclases o interfaces fuera del paquete.
2. **Buenas prácticas en el constructor:** Para no repetir código de validación, puedes llamar a los setters dentro del constructor en lugar de duplicar el if-else:

```
public Entidad(String nombre, double energia, int edad, boolean viva){
    this.nombre = nombre;
    setEnergia(energia); // Aplica tu validacion de energia
    setEdad(edad);      // Aplica tu validacion de edad
    this.viva = viva;
}
```

El resto está de 10. Pasemos a la siguiente: ¿Animal.java, Mortal.java o Estadisticas.java?

User prompt: Animal.java: package ecosim; abstract class Animal extends Entidad implements Mortal{ //ATRIBUTOS //Heredados: String nombre, double energia, int edad,
 boolean viva private int velocidad; private double peso; //CONSTRUCTOR public Animal(String nombre, double energia, int edad, boolean viva, int velocidad, double
 peso){ super(nombre, energia, edad, viva); this.velocidad = velocidad; this.peso = peso; } //GETS public int getVelocidad(){ return this.velocidad; }
 public double getPeso(){ return this.peso; } //SETS public void setVelocidad(int velocidad){ this.velocidad = velocidad; } public void setPeso(double
 peso){ this.peso = peso; } //METODOS abstract void comer(Ecosistema eco); public void moverse(){ System.out.println(this.getNombre() + " se movio"); }}

Response: Tu clase Animal.java está casi perfecta y muy bien estructurada. Para que funcione sin fallas de compilación ni problemas de herencia con las subclases (Conejo y Lobo), ajusta únicamente los siguientes detalles:

1. **Visibilidad public:** Agrega el modificador public a la clase (public abstract class Animal), al método abstracto (public abstract void comer(Ecosistema eco)) y haz que las subclases puedan heredarlos sin restricciones.

2. Implementación de la interfaz Mortal: Dado que declaraste `implements Mortal`, esta clase (o las subclases) debe implementar obligatoriamente los métodos `estaVivo()` y `morir()` definidos en la interfaz. Si los implementas directamente en `Animal`, `Conejo` y `Lobo` ya los heredarán automáticamente sin repetir código.

Versión corregida y completa de `Animal.java`

```
package ecosim;

public abstract class Animal extends Entidad implements Mortal {
    // ATRIBUTOS
    // Heredados: String nombre, double energia, int edad, boolean viva
    private int velocidad;
    private double peso;

    // CONSTRUCTOR
    public Animal(String nombre, double energia, int edad, boolean viva, int velocidad, double peso) {
        super(nombre, energia, edad, viva);
        this.velocidad = velocidad;
        this.peso = peso;
    }

    // METODO ABSTRACTO
    public abstract void comer(Ecosistema eco);

    // METODO CONCRETO
    public void moverse() {
        System.out.println(this.getNombre() + " se movio.");
    }

    // IMPLEMENTACION DE INTERFAZ MORTAL (heredada por Conejo y Lobo)
    @Override
    public boolean estaVivo() {
        return getViva() && getEnergia() > 0;
    }

    @Override
    public void morir() {
        setViva(false);
        setEnergia(0);
        System.out.println(getNombre() + " ha muerto.");
    }

    // GETTERS
    public int getVelocidad() {
        return this.velocidad;
    }
}
```

```

    public double getPeso() {
        return this.peso;
    }

    // SETTERS
    public void setVelocidad(int velocidad) {
        this.velocidad = velocidad;
    }

    public void setPeso(double peso) {
        this.peso = peso;
    }
}

```

¡Excelente trabajo! ¿Pasamos a la interfaz `Mortal.java` o a la clase de `Estadisticas.java`?

User prompt: genial, aqui te paso Mortal: package ecosim; public interface Mortal { public boolean estaVivo(); public void morir(); }

Response: ¡Vas excelente! La interfaz Mortal está casi terminada.

Solo le falta agregar el **método default** `verificarMuerte()`, que es un requisito obligatorio del enunciado para aprovechar esta característica de Java 8+. Este método evalúa si la entidad se quedó sin energía y, de ser así, llama automáticamente a `morir()`.

Código corregido y completo para `Mortal.java`

```

package ecosim;

public interface Mortal {
    // Métodos abstractos
    boolean estaVivo();
    void morir();

    // Método default requerido por la consigna
    default void verificarMuerte() {
        if (!estaVivo()) {
            morir();
        }
    }
}

```

¿Por qué es útil este método default?

El método default te permite incluir lógica con cuerpo dentro de una interfaz sin romper el diseño de POO. Así, desde cualquier lugar del loop de simulación (por ejemplo, después de que un animal coma, actúe o envejezca), puedes simplemente hacer:

```
animal.verificarMuerte();
```

Y la interfaz se encargará sola de verificar la energía y ejecutar el proceso de muerte si corresponde.

¿Te parece que pasemos ahora a `Estadisticas.java` para dejar completada toda tu parte asignada?

User prompt: genial, de Estadisticas no tengo nada hecho porque en si no sabia si es una clase, una interface o una simple funcion en la ejecucion del codigo

Response: Es completamente comprensible la duda. En programación orientada a objetos (POO), la mejor práctica cuando tienes que llevar el control de un historial a lo largo del tiempo es **crear una clase dedicada** (`Estadisticas.java`).

Si usaras solo variables o funciones sueltas dentro de la ejecución, el código de la simulación te quedaría sobrecargado, desordenado y perderías la capacidad de consultar los datos históricos fácilmente.

Crear la clase `Estadisticas.java` te permite instanciar un objeto dentro de `Ecosistema` que **se encarga exclusivamente de registrar y procesar las poblaciones turno a turno**.

Código listo para copiar: `Estadisticas.java`

```
package ecosim;

import java.util.ArrayList;

public class Estadisticas {

    // Clase interna auxiliar para guardar la "foto" de cada turno
    private static class RegistroTurno {
        private int turno;
        private int cantPlantas;
        private int cantConejos;
        private int cantLobos;

        public RegistroTurno(int turno, int cantPlantas, int cantConejos, int cantLobos) {
            this.turno = turno;
            this.cantPlantas = cantPlantas;
            this.cantConejos = cantConejos;
            this.cantLobos = cantLobos;
        }
    }

    // Lista dinámica para almacenar el historial de turnos
    private ArrayList<RegistroTurno> historial;

    // Constructor
    public Estadisticas() {
        this.historial = new ArrayList<>();
    }
}
```

```

}

// Método para registrar la cantidad de entidades al finalizar cada turno
public void registrarTurno(int turno, int plantas, int conejos, int lobos) {
    historial.add(new RegistroTurno(turno, plantas, conejos, lobos));
}

// Método para calcular y mostrar los máximos y mínimos requeridos en el BONUS
public void mostrarReporteEstadistico() {
    if (historial.isEmpty()) {
        System.out.println("No hay datos estadísticos registrados.");
        return;
    }

    // Inicializamos los punteros con el primer turno registrado
    RegistroTurno maxP = historial.get(0), minP = historial.get(0);
    RegistroTurno maxC = historial.get(0), minC = historial.get(0);
    RegistroTurno maxL = historial.get(0), minL = historial.get(0);

    // Recorremos todo el historial para evaluar los picos máximos y mínimos
    for (RegistroTurno r : historial) {
        // Plantas
        if (r.cantPlantas > maxP.cantPlantas) maxP = r;
        if (r.cantPlantas < minP.cantPlantas) minP = r;

        // Conejos
        if (r.cantConejos > maxC.cantConejos) maxC = r;
        if (r.cantConejos < minC.cantConejos) minC = r;

        // Lobos
        if (r.cantLobos > maxL.cantLobos) maxL = r;
        if (r.cantLobos < minL.cantLobos) minL = r;
    }

    System.out.println("\n=====");
    System.out.println("    BONUS: ESTADÍSTICAS EN TIEMPO REAL (HISTORIAL)    ");
    System.out.println("=====");
    System.out.println("PLANTAS:");
    System.out.println(" * Maximo: " + maxP.cantPlantas + " plantas en el Turno " + maxP.turno);
    System.out.println(" * Minimo: " + minP.cantPlantas + " plantas en el Turno " + minP.turno);

    System.out.println("\nCONEJOS:");
    System.out.println(" * Maximo: " + maxC.cantConejos + " conejos en el Turno " + maxC.turno);
    System.out.println(" * Minimo: " + minC.cantConejos + " conejos en el Turno " + minC.turno);

    System.out.println("\nLOBOS:");

```

```

        System.out.println(" * Maximo: " + maxL.cantLobos + " lobos en el Turno " + maxL.turno);
        System.out.println(" * Minimo: " + minL.cantLobos + " lobos en el Turno " + minL.turno);
        System.out.println("=====\\n");
    }
}

```

¿Cómo lo integra tu compañero en la clase Ecosistema?

Quien haga la clase Ecosistema solo debe incluir dos líneas de código para vincular tu trabajo:

1. Crear el objeto en Ecosistema:

```
private Estadisticas estadisticas = new Estadisticas();
```

2. Llamarlo al final de cada turno y al terminar la simulación:

```

// Al finalizar cada turno dentro de procesarTurno():
estadisticas.registrarTurno(turnoActual, plantas.size(), conejos.size(), lobos.size());

// Al momento de dar el reporte final:
estadisticas.mostrarReporteEstadistico();

```

¡Con esto ya tienes tus **4 componentes requeridos** (Entidad, Animal, Mortal y Estadisticas) terminados, validados e integrados según las reglas de POO! ¿Quieres que revisemos cómo se conectan con el resto de las clases de tus compañeros o te queda alguna otra consulta?
